

Dynamic Load Balancing and Persistent Chat

CSD Course — Lab Assignment 6

Student Name	Rahul Raj
Roll Number	12341680
Date	04 September 2026
Public URL	http://10.1.75.53:3269 (dynamic load balancer on sys1; dashboard at /lb/)
Source code	https://github.com/Rahul5977/csd-lab6-dynamic-lb-persistent-chat

1. Assigned Systems and Ports

Role	Alias	SSH	Container IP	Service port(s)	External port
Load balancer + database service	stu78_sys1	ssh -p 2269 student@10.1.75.53	172.17.0.70	LB 3000 · SQLite DB service 5270	3269 → LB (the only URL clients use)
Backend A	stu78_sys2	ssh -p 2270 student@10.1.75.53	172.17.0.71	app 3000	3270 (direct, kept for the previous assignment)
Backend B	stu78_sys3	ssh -p 2271 student@10.1.75.53	172.17.0.72	app 3000	3271 (direct, previous assignment)
Backend C	stu78_sys4	ssh -p 2272 student@10.1.75.53	172.17.0.73	app 3001	— (port 3000 / external 3272 belongs to my course project)

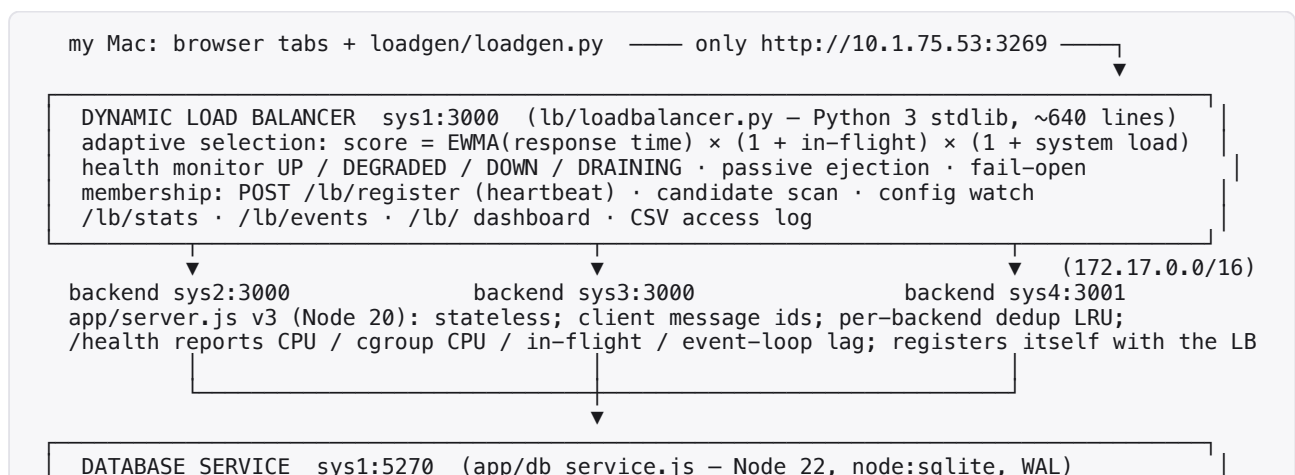
The four systems are Ubuntu 24.04 containers on one shared lab host, each limited by cgroup to **exactly one CPU core**. The lab NAT forwards only the first external port of each system to container port 3000. Clients — the browser and the load generator on my Mac — talk only to `10.1.75.53:3269`; the load balancer reaches the backends over the private bridge network, so a backend does not need a public port at all (which is why sys4's backend can live on 3001). The load generator runs on my local machine.

2. Objective

Extend the previous secure group-chat application and deploy its backend on at least three allotted systems behind a load balancer. The load balancer must select a backend **dynamically** using response time and/or current system load, monitor backend health and stop sending to unavailable backends, and **automatically detect backends started while the application is running**. All backends must share **persistent** chat data, every message must have a **unique id**, and duplicate insertion must be prevented on retries/reconnections. Generate increasing load, record response time, throughput, successful/failed requests and the number of active backends, and analyse the results.

Everything above is implemented, deployed on the allotted systems, measured, and demonstrated in this report. The base is my Assignment-5 system (Python load balancer, Node.js chat backend with the Assignment-4 end-to-end encryption); Assignment 6 adds a second-generation balancer, a real database, and idempotent messaging. The previous assignment keeps working: its files and directories on the lab systems are untouched and `scripts/rollback_lab5.sh` restores its processes in twenty seconds.

3. Architecture



```
users · sessions · rooms · messages(id PRIMARY KEY, UNIQUE(room,seq)) · dedup_log
POST /messages is idempotent (INSERT ... ON CONFLICT(id) DO NOTHING in one transaction)
WebSocket firehose → every backend delivers new messages to its own connected clients
```

Request lifecycle. A request arrives at the balancer, which picks a backend with the adaptive algorithm (§4), adds `X-Forwarded-For` / `X-Real-IP` and proxies it over a pooled keep-alive connection. The backend validates the session (30 s local cache, source of truth in the database), performs the operation against the database service and answers with `X-Backend-Id` (which instance served it) — the balancer adds `X-LB-Active-Backends` (how many backends were routable at that moment); the load generator records both on every request. A new message is appended once to SQLite; the database broadcasts it on a WebSocket firehose and every backend pushes it to its own connected browsers, so three stateless backends behave as one application. Browser WebSockets are pinned to a backend by IP-hash and tunnelled byte-for-byte.

What is inherited from Assignments 4/5 unchanged: script password hashing with constant-time compare and login rate limiting; end-to-end encrypted "locked" rooms (PBKDF2-HMAC-SHA-256 with 250 000 iterations → AES-256-GCM, fresh IV per message, AAD binding room and key epoch, key derived in the browser — the server stores only ciphertext); the WebSocket delivery path; the clean single-page UI.

4. Load Balancer Design (sys1)

`lb/loadbalancer.py` — pure Python 3 standard library (key code in §5, full source on GitHub). It is the Assignment-5 balancer extended in four areas.

4.1 Dynamic backend selection

Every backend carries live bookkeeping: an **exponentially weighted moving average of its response time** (`ewma_ms` , $\alpha = 0.2$, fed by every proxied request *and* by the health probe's own latency, so an idle backend keeps a fresh estimate), its **in-flight request count**, and the **system load it reports** in `/health` (process CPU %, container-wide cgroup CPU % against its quota, 1-minute load average, event-loop lag). The score is

```
score(b) = EWMA_rt(b) × (1 + in_flight(b)) × (1 + load_weight × cpu_load(b)) / weight(b)
          (× 2 while the backend is DEGRADED; 0 for a backend that has never been sampled)
```

and the `adaptive` algorithm (default) uses **power-of-two-choices**: sample two backends at random, keep the lower score. This is the standard remedy for the herding problem of a pure "pick the minimum" rule — with sequential traffic every backend has zero in-flight requests and a pure minimum sends everything to whichever backend happened to have the lowest EWMA; with two random choices, equal backends split evenly while a slower or more loaded backend loses every pairing it is drawn into and is left with about $1/N^2$ of the traffic until its score improves. A never-sampled backend scores 0, so a **newly added backend receives traffic immediately**; a 5 % exploration share keeps every backend measured. `least_response_time` (pure minimum, no load term) and the four Assignment-5 algorithms (`round_robin` , `least_connections` , `weighted_round_robin` , `ip_hash`) remain selectable — the algorithm comparison in §9 uses that.

4.2 Health monitoring

A background thread probes every backend's `/health` every 3 s (6 s timeout) and reads the load JSON. States instead of a boolean:

State	Meaning	Routable?
UP	answering in time	yes
DEGRADED	answering, but probe slower than 2 s or reported CPU ≥ 95 %	yes, score doubled
DOWN	connection refused / timeouts for 2 consecutive checks; back to UP after 2 successes	no

State	Meaning	Routable?
DRAINING	asked to leave (<code>/lb/deregister</code>): finishes in-flight, gets nothing new	no

Passive checks eject a backend immediately when a live proxy attempt cannot connect; the request is then retried on another backend (only if nothing was sent upstream yet — a POST can never be duplicated by the balancer, and even if it were, the message id makes it harmless). **Fail-open:** the last routable backend is never ejected — a slow-but-alive backend beats an empty pool. "Slow $\neq$ dead" fixes the ejection churn I diagnosed in Assignment 5 (a saturated 1-core backend answered its health check late, was ejected, and the survivors collapsed in turn).

4.3 Dynamic membership — detecting backends started while running

Three mechanisms, any one of which is sufficient:

1. **Self-registration (push).** Each backend POSTs `/lb/register {id, host, port, weight}` with a shared token at boot and every 5 s as a heartbeat. An unknown backend is admitted on the spot (`added` event). On SIGTERM the backend POSTs `/lb/deregister` and drains — graceful scale-down.
2. **Candidate scan (pull).** `lb.conf.json` lists candidate slots (sys3:3000, sys4:3001). Every 5 s the balancer probes the ones it does not know; any that answers `/health` with the expected application version is admitted (`source = scan`). This found a backend even when the backend's own registration was disabled — and, during development, it correctly *refused* the old Assignment-5 backend still running on sys3 because it reports a different version.
3. **Config watch.** The configuration file is re-read whenever its modification time changes (also `SIGHUP` and `POST /lb/reload`); `POST /lb/config` switches the algorithm or tuning at runtime.

Dynamically discovered backends that stay DOWN for 15 min are pruned; statically configured ones are kept as DOWN.

4.4 Observability

`/lb/stats` (per-backend state, source, score, EWMA, probe latency, load, in-flight, counts, percentiles, plus `active_backends`), `/lb/events` (timeline of `added` / `degraded` / `ejected` / `readmitted` / `draining` / `removed` / config events), the auto-refreshing dashboard at `/lb/` (screenshot §11), and a CSV access log with the active-backend count on every line.

5. Load Balancer — Key Code

The complete balancer (`lb/loadbalancer.py` , ~800 lines) is in the repository linked on the cover page. The two parts that make it *dynamic* are the score and the selection:

```

1  def cpu_load(self):
2      """0..1 : how busy the backend's SYSTEM is. Prefers the container-wide
3      cgroup CPU utilisation (sys_cpu_pct — sees other tenants of the box),
4      falls back to the process CPU % and the load average per core."""
5      cpu = float(self.load.get("cpu_pct", 0) or 0) / 100.0
6      sys_cpu = self.load.get("sys_cpu_pct")
7      if sys_cpu is not None:
8          cpu = max(cpu, float(sys_cpu) / 100.0)
9      cores = max(1, int(self.load.get("cores", 1) or 1))
10     la = float(self.load.get("loadavg1", 0) or 0) / cores
11     return max(cpu, min(la, 4.0))
12
13  def score(self, cfg):
14      """Lower is better. New backends (no sample) score 0 -> tried at once."""
15      if self.ewma_ms is None:
16          return 0.0
17      s = self.ewma_ms * (1 + self.in_flight)
18      if cfg["algorithm"] == "adaptive":
19          s *= 1 + cfg["load_weight"] * self.cpu_load()
```

```

20     if self.state == DEGRADED:
21         s *= 2.0
22     return s / max(1, self.weight)

```

```

1     def pick(self, client_ip, exclude=()):
2         """Choose a backend for this request using the configured algorithm."""
3         pool = [b for b in self.routable_backends() if b.id not in exclude]
4         if not pool:
5             return None
6         algo = self.cfg["algorithm"]
7         with self.lock:
8             if algo in ("adaptive", "least_response_time"):
9                 fresh = [b for b in pool if b.ewma_ms is None]
10                if fresh:
11                    # never-sampled backend: try it now
12                    return fresh[0]
13                if len(pool) > 1 and random.random() * 100 < self.cfg["explore_pct"]:
14                    # exploration keeps estimates fresh
15                    return random.choice(pool)
16                if algo == "least_response_time":
17                    # pure minimum (herds under light load)
18                    return min(pool, key=lambda b: (b.score(self.cfg), b.in_flight, b.requests))
19                # adaptive = "power of two choices": sample two backends at random and
20                # keep the better score. Equal backends split evenly (no herding onto
21                # one lucky EWMA); a slow or loaded backend loses every pairing and is
22                # left with ~1/N² of the traffic until its score improves.
23                a, b = random.choice(pool), random.choice(pool)
24                return min((a, b), key=lambda x: (x.score(self.cfg), x.in_flight, x.requests))
25            if algo == "least_connections":
26                return min(pool, key=lambda b: (b.in_flight, b.requests))
27            if algo == "ip_hash":
28                return pool[hash(client_ip) % len(pool)]
29            if algo == "weighted_round_robin":
30                total = sum(b.weight for b in pool)
31                for b in pool:
32                    b.current_weight += b.weight
33                chosen = max(pool, key=lambda b: b.current_weight)
34                chosen.current_weight -= total
35                return chosen
36            # default: round_robin
37            self.rr_index = (self.rr_index + 1) % len(pool)
38            return pool[self.rr_index]

```

6. Database Persistence and Duplicate Prevention

6.1 Persistent shared storage

All backends are stateless; every user, session, room and message lives in **one SQLite database (WAL mode)** owned by `app/db_service.js` on `sys1` and reached over HTTP by every backend. The database file (`~/assignment6/data/chat.sqlite`) survives restarts of any backend, of the database service, and of the whole cluster (verified live in §9.4). Assignment 5's data (717 users, 5 rooms, 111 180 messages) was migrated in on first boot, so the old accounts still work. SQLite was chosen because the lab boxes allow no root and no package installation; Node 22's built-in `node:sqlite` needs nothing but a user-local Node binary, and a real relational engine gives real constraints.

6.2 Unique message ids

Every message has a **UUID v4 id minted by the client when the message is composed** and reused on every retry (body field `id` or `Idempotency-Key` header). Only the client knows that two sends are "the same message" — a server-minted id cannot recognise a retry — so this is the right place for the id. A client that sends none gets one minted by the backend (still unique; but then a retry is, by definition, a new message).

6.3 How duplicates are prevented — three layers

Layer	Where	Mechanism
1 · fast path	each backend	LRU of the last 5 000 stored ids (also fed by the firehose, so a retry that lands on a <i>different</i> backend is usually caught here too) → <code>200 {duplicate:true}</code> without touching the database

Layer	Where	Mechanism
2 · authoritative	database	messages.id TEXT PRIMARY KEY ; POST /messages runs <code>SELECT ... ; INSERT ... ON CONFLICT(id) DO NOTHING</code> inside one <code>BEGIN IMMEDIATE</code> transaction. Two backends sending the same id at the same instant are serialised; the loser gets the stored row back with <code>duplicate:true</code> and the rejection is written to <code>dedup_log</code>
3 · presentation	browser	renders each id once; a retried send that the server deduplicated never appears twice

A duplicate is answered with **HTTP 200**, `duplicate:true`, the original seq and id, and header **X-Duplicate: 1** — an idempotent success, never an error the client would retry again. Per-room ordering uses `UNIQUE(room, seq)` so sequences are gap-free. Schema excerpt and the append transaction:

```

1 // — Idempotent append: the heart of the "no duplicates" guarantee —————
2 // Runs as ONE transaction. Node is single-threaded and DatabaseSync is
3 // synchronous, so two backends POSTing the same id at the same instant are
4 // serialised here; the second one finds the row and is reported as duplicate.
5 const appendTx = (room, entry) => {
6   db.exec('BEGIN IMMEDIATE');
7   try {
8     const existing = q.getMsg.get(entry.id);
9     if (existing) {
10      q.logDup.run(entry.id, room, entry.via || null, Date.now());
11      db.exec('COMMIT');
12      return { duplicate: true, rec: rowToEntry(existing) };
13    }
14    const seq = q.nextSeq.get(room).seq;
15    const extra = {};
16    for (const k of Object.keys(entry)) {
17      if (!['id', 'seq', 'room', 'from', 'ts', 'kind', 'text', 'env', 'via'].includes(k)) extra[k] =
entry[k];
18    }
19    const res = q.insMsg.run(entry.id, room, seq, entry.from, entry.ts || Date.now(), entry.kind,
entry.kind === 'enc' ? null : (entry.text ?? null),
20    entry.kind === 'enc' ? JSON.stringify(entry.env) : null,
21    entry.via || null, Date.now(), Object.keys(extra).length ? JSON.stringify(extra) : null);
22    if (res.changes === 0) { // lost a race inside the same tx window
23      const row = q.getMsg.get(entry.id);
24      q.logDup.run(entry.id, room, entry.via || null, Date.now());
25      db.exec('COMMIT');
26      return { duplicate: true, rec: rowToEntry(row) };
27    }
28    db.exec('COMMIT');
29    return { duplicate: false, rec: rowToEntry(q.getMsg.get(entry.id)) };
30  } catch (e) {
31    db.exec('ROLLBACK');
32    throw e;
33  }
34 }
35 };

```

6.4 Verification through the public URL

`scripts/dedup_test.py` verifies against the **database row count**, not against what a backend said:

Test	Scenario	Sent	Stored in DB	duplicate:true responses	Backends involved	Result
T1	same id, sequential retries	20	1	19	sys2, sys3, sys4	PASS
T2	same id, concurrent storm across backends	20	1	19	sys2, sys3, sys4	PASS
T3	send, drop connection, reconnect, re-send	2	1	1	sys3, sys4	PASS
T4	Idempotency-Key header	5	1	4	sys4	PASS
T5	control: distinct ids	20	20	0	sys3	PASS
P1	persistence across DB-service restart	49	49	1	via LB	PASS: count 48→49 across restart, probe readable=True, dup-of-old-id still rejected=True

Database after the test: 273769 messages, 10 duplicates rejected in total (sqlite (node:sqlite, WAL)).

The 20-way concurrent storm is the important one: twenty connections, spread by the balancer over all three backends, POST the same id at the same instant; exactly one row exists afterwards and all twenty responses carry the same seq. Test P1 shows that a duplicate of an id stored *before* a database restart is still rejected (the guard is durable, not in memory). The browser's "Resend last" button — which re-POSTs the previous message with the same id — is answered with `duplicate:true` (screenshot §11).

7. Load Generator and Experimental Method

`loadgen/loadgen.py` (Python 3 stdlib, on my Mac) drives the public URL with a realistic mix per virtual user — 10 % login (script, the expensive one), 55 % fetch history, 30 % send, 5 % health. Every send carries a client-minted id and is retried with the same id on a transport error, exactly like the browser. Modes: **closed loop** (N users in a request→response loop), **open loop** (Poisson arrivals at a fixed *offered* rate, independent of how slow responses get), and **ramp** (a schedule of concurrency steps). Per request it records latency, status, error class, `X-Backend-Id`, `X-LB-Active-Backends` and the duplicate flag; with `--poll-stats` it samples `/lb/stats` once a second, giving the number of active backends, per-backend state, score and CPU as a time series. The first 5 s of every run are discarded; percentiles are computed over the full sample.

Experiment	What varies	Held constant
L response time vs load	concurrency 1, 10, 25, 50, 100, 200 × backends 1, 2, 3 (2 reps, 40 s)	adaptive algorithm; configurations interleaved in shuffled order inside every (level, rep) round so 1/2/3-backend comparisons share the same host conditions
O throughput vs offered load	offered 20, 50, 100, 200, 300 req/s (Poisson) × backends 1, 2, 3 (30 s)	as above
SCALE dynamic scaling	one 300 s run: 25 → 50 → 100 → 200 users; sys3 started at ~128 s and sys4 at ~184 s <i>while the run was going</i>	adaptive
FAIL failure / recovery	150 s at 100 users on 3 backends; sys3 SIGKILLED at ~30 s, restarted at ~90 s	adaptive
ALGO dynamic vs static	adaptive vs round_robin vs least_connections at 50 users while a CPU-burning process runs on sys3 (2 reps)	3 backends

The number of backends is always changed the *dynamic* way — by starting or stopping the backend process on the system (`scripts/scale.sh sysN up|down|kill`) — never by editing the balancer's configuration; the balancer must notice by itself.

8. Results — Response Time vs Load and Throughput vs Offered Load

8.1 Response time vs load (closed loop)

Concurrency	Backends	Total	Success	Failed	Err %	Throughput (req/s)	Mean ms	p50	p90	p95	p99	Active	Distribution	Speed up vs 1
1	1	1610	1610	0	0.0	45.99	21.88	11.63	72.37	95.19	134.33	1-1	sys2:100.0%	—
1	2	1538	1538	0	0.0	43.95	22.93	11.98	76.28	97.22	146.66	2-2	sys2:51.0% / sys3:49.0%	0.96×
1	3	1521	1521	0	0.0	43.46	23.16	12.62	75.34	97.06	145.57	3-3	sys2:31.4% / sys3:30.3% / sys4:38.3%	0.94×
10	1	3339	3339	0	0.0	95.4	105.78	68.07	238.59	554.72	840.49	1-1	sys2:100.0%	—
10	2	5736	5736	0	0.0	163.89	61.55	23.88	152.65	267.81	460.72	2-2	sys2:49.0% / sys3:51.0%	1.72×
10	3	7874	7874	0	0.0	224.97	44.75	15.25	122.47	197.3	331.12	3-3	sys2:33.7% / sys3:33.8% / sys4:32.4%	2.36×

Concurrency	Backends	Total	Success	Failed	Err %	Throughput (req/s)	Mean ms	p50	p90	p95	p99	Active	Distribution	Speed up vs 1
25	1	3576	3576	0	0.0	102.16	248.31	89.42	378.24	1704.71	2181.48	1-1	sys2:100.0%	—
25	2	6244	6244	0	0.0	178.42	141.7	69.13	328.54	749.59	1122.3	2-2	sys2:49.9% / sys3:50.1%	1.75x
25	3	9116	9116	0	0.0	260.47	96.99	51.0	253.09	410.63	657.77	3-3	sys2:34.5% / sys3:32.3% / sys4:33.1%	2.55x
50	1	3108	3108	0	0.0	88.81	553.3	80.05	335.24	5186.82	5907.09	1-1	sys2:100.0%	—
50	2	6203	6203	0	0.0	177.23	288.56	105.31	1036.97	1809.18	2366.83	2-2	sys2:48.0% / sys3:52.0%	2.00x
50	3	9618	9618	0	0.0	274.78	183.8	86.37	373.44	982.51	1470.75	3-3	sys2:34.4% / sys3:33.1% / sys4:32.4%	3.09x
100	1	3607	3607	0	0.0	103.06	895.99	110.51	784.04	8976.44	10533.22	1-1	sys2:100.0%	—
100	2	5972	5972	0	0.0	170.63	587.32	168.87	959.22	4402.06	5299.23	2-2	sys2:49.5% / sys3:50.5%	1.66x
100	3	9429	9429	0	0.0	269.4	377.63	126.6	592.07	2587.86	3441.87	3-3	sys2:33.2% / sys3:33.9% / sys4:32.9%	2.61x
200	1	2970	2970	0	0.0	84.85	1384.8	772.25	1356.95	7598.55	12532.98	1-1	sys2:100.0%	—
200	2	6632	6632	0	0.0	189.49	959.87	260.11	1541.22	7855.05	10046.2	2-2	sys2:48.7% / sys3:51.3%	2.23x
200	3	9378	9378	0	0.0	267.93	736.58	216.81	1398.91	5716.81	7009.31	3-3	sys2:32.6% / sys3:34.3% / sys4:33.1%	3.16x

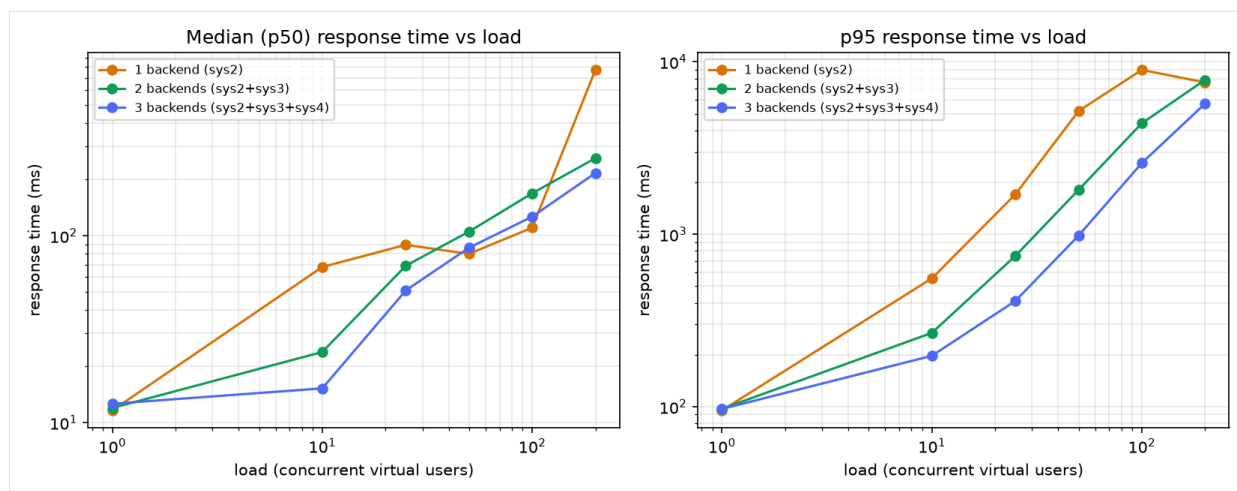


Figure 1 — Response time vs load (p50 left, p95 right), 1 / 2 / 3 backends, log-log.

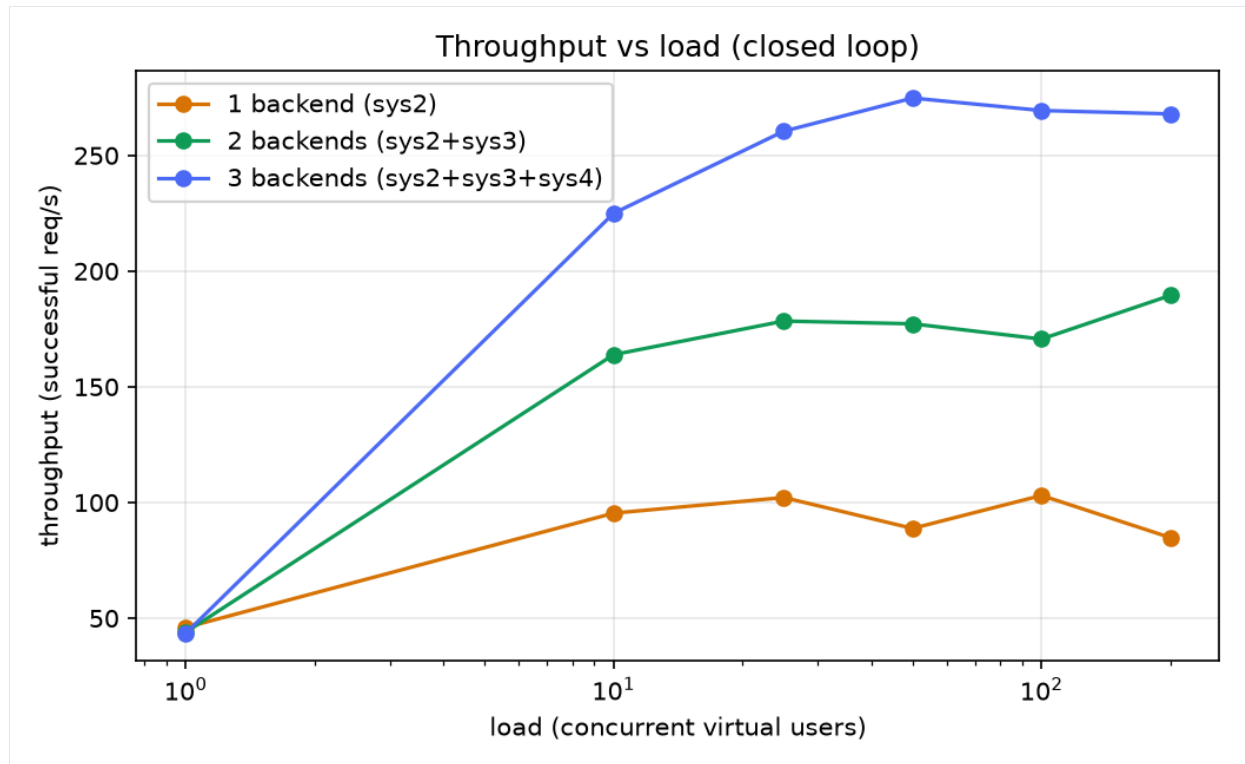


Figure 2 — Throughput vs load (closed loop).

Reading the table. At one user the three configurations are identical (44–46 req/s, p50 $\approx$ 12 ms): a single sequential client is bound by its own round trip Mac → lab → Mac, and no number of backends can shorten one request. From ten users upward the backends are the bottleneck and the balancer's value is plain: at 10 users **95 → 164 → 225 req/s** ($1.7\times / 2.4\times$) with p95 falling from 555 ms to 197 ms; at 50 users **89 → 177 → 275 req/s** ($2.0\times / 3.1\times$) with p95 5.2 s → 1.0 s; at 200 users **85 → 189 → 268 req/s** ($2.2\times / 3.2\times$) and median response time 772 ms → 217 ms. A single 1-core backend saturates at $\sim$ 90–105 req/s regardless of how many users push it — everything beyond that becomes queueing delay (p95 reaches 9 s at 100 users) — while two and three backends move the ceiling to $\sim$ 180 and $\sim$ 270 req/s. The speed-ups slightly above 3 $\times$ at 50 and 200 users are within run-to-run noise on the shared host (the single-backend runs at those levels happened to land in busier minutes); the honest summary is *near-linear scaling, 2.4–3.2 $\times$ for three backends*. Every run had **zero failed requests**, the adaptive algorithm split traffic 49/51 % and 33/34/33 % (it favours nobody when the backends are equal), and `Active` confirms the balancer saw exactly the intended number of backends throughout each run.

8.2 Throughput vs offered load (open loop)

Offered load nominal → actual (req/s)	Backends	Achieved (req/s)	Success	Failed (of which dropped)	Err %	p50 ms	p95 ms	p99 ms	Active	Distribution
20 → 19.2	1	19.2	480	0 (0)	0.0	23.13	111.11	168.21	1-1	sys2:100.0%
20 → 20.3	2	20.28	507	0 (0)	0.0	23.36	112.24	176.71	2-2	sys2:37.3% / sys3:62.7%
20 → 17.6	3	17.64	441	0 (0)	0.0	27.93	129.19	197.92	3-3	sys2:18.1% / sys3:40.8% / sys4:41.0%
50 → 44.6	1	44.6	1115	0 (0)	0.0	24.83	170.01	339.36	1-1	sys2:100.0%
50 → 43.6	2	43.6	1090	0 (0)	0.0	23.6	135.41	193.85	2-2	sys2:40.8% / sys3:59.2%
50 → 41.2	3	41.16	1029	0 (0)	0.0	23.29	117.69	169.77	3-3	sys2:25.7% / sys3:34.2% / sys4:40.1%
100 → 76.4	1	76.44	1911	0 (0)	0.0	38.99	215.17	768.78	1-1	sys2:100.0%

Offered load nominal → actual (req/s)	Backends	Achieved (req/s)	Success	Failed (of which dropped)	Err %	p50 ms	p95 ms	p99 ms	Active	Distribution
100 → 79.9	2	79.88	1997	0 (0)	0.0	24.96	159.79	278.7	2-2	sys2:42.1% / sys3:57.9%
100 → 80.5	3	80.48	2012	0 (0)	0.0	25.77	148.35	240.83	3-3	sys2:26.4% / sys3:38.4% / sys4:35.2%
200 → 137.9	1	137.92	3448	0 (0)	0.0	352.22	8948.85	12706.19	1-1	sys2:100.0%
200 → 148.7	2	148.68	3717	0 (0)	0.0	42.08	230.62	417.61	2-2	sys2:48.9% / sys3:51.1%
200 → 145.9	3	145.92	3648	0 (0)	0.0	27.2	181.08	291.68	3-3	sys2:30.5% / sys3:36.5% / sys4:33.1%
300 → 200.2	1	118.64	2966	2038 (797)	40.73	715.03	7422.47	16136.79	1-1	sys2:59.3%
300 → 205.5	2	205.48	5137	0 (0)	0.0	283.48	4488.56	6321.74	2-2	sys2:46.1% / sys3:53.9%
300 → 220.7	3	220.68	5517	0 (0)	0.0	64.08	267.6	415.81	3-3	sys2:33.3% / sys3:33.9% / sys4:32.8%

Actual offered load = arrivals the generator really issued per second (successful + failed + dropped). The thread-per-arrival Python generator on the Mac saturates near 230 arrivals/s, so the nominal 300 req/s setting produced ~220–230 req/s of real offered load.

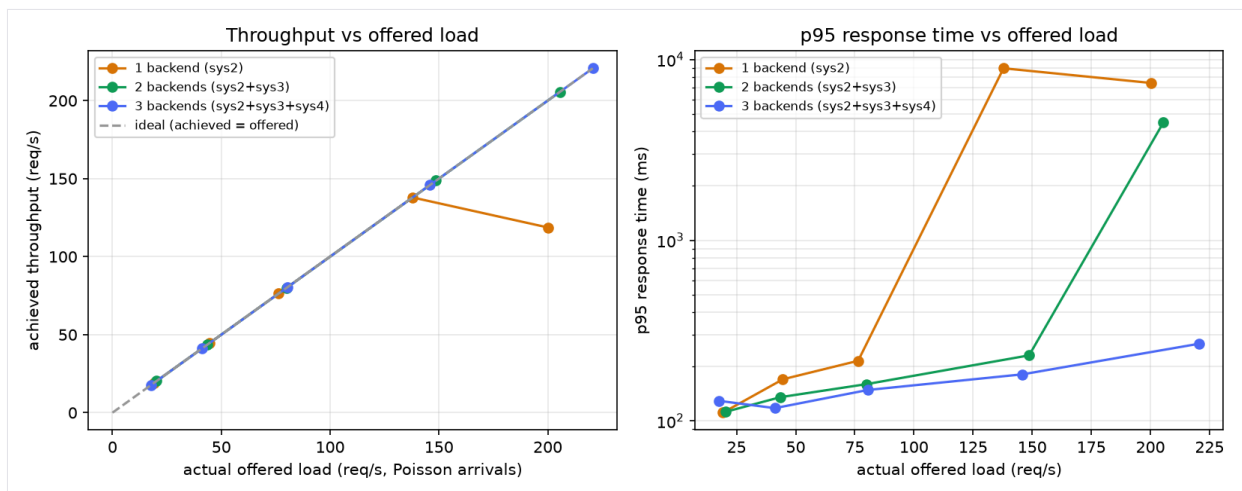


Figure 3 — Throughput vs offered load (open loop, Poisson arrivals); dashed = ideal.

Reading the table. In open-loop mode arrivals come at a fixed rate whether or not the system keeps up, so the *achieved* curve bends away from the ideal line exactly where the system saturates (Figure 3, left) and response time explodes just before it (right). Up to ~80 req/s of real offered load all three configurations keep up — throughput follows the ideal line and p95 stays at 110–215 ms. At ~140 req/s the single backend is already over its knee: p50 352 ms and **p95 8.9 s** (a queue that never drains) while two and three backends answer in 231 ms and 181 ms at p95. At ~230 req/s the single backend collapses — **119 req/s achieved, 41 % of arrivals failed** (2 038, of which 797 were shed by the generator's in-flight cap because responses no longer came back), p95 7.4 s — two backends just cope (205 req/s, 0 failed, but p95 4.5 s: at capacity, queue building) and three backends absorb it comfortably (**221 req/s, 0 failed, p95 268 ms**). The offered-load axis uses what the generator actually issued: its thread-per-arrival Python implementation tops out at ~230 arrivals/s, which is why the nominal 300 req/s point is plotted at ~230. Successful/failed request counts per point are in the table; the number of active backends was constant within each run (`Active` column).

9. Results — Dynamic Scaling, Failure Recovery, Algorithm Comparison

9.1 Dynamic scaling: performance after each backend is added

The system started with **one** backend (sys2). Load was raised in steps (25 → 50 → 100 users) until the single backend was saturated; then `bash scripts/scale.sh sys3 up` and later `... sys4 up` were executed on the other systems *while the load generator kept running*. The balancer admitted each new backend by itself (registration heartbeat and candidate scan both fired) and the load generator's per-second samples of `/lb/stats` show exactly when.

Phase	Window (s)	Users	Active backends	Throughput (req/s)	p50 ms	p95 ms	Errors	Share sys2 / sys3 / sys4
A: 1 backend, 25 users	5–40	25	1	95.5	75	2089	0	100% / 0% / 0%
B: 1 backend, 50 users	45–80	50	1	105.2	75	4482	0	100% / 0% / 0%
C: 1 backend, 100 users (saturated)	85–128	100	1	98.6	61	9028	0	100% / 0% / 0%
D: sys3 added → 2 backends	136–184	100	2	189.5	100	4172	0	52% / 47% / 1%
E: sys4 added → 3 backends	192–260	100	3	285.2	102	2432	0	34% / 34% / 32%
F: 3 backends, 200 users	265–300	200	3	291.1	133	5710	0	34% / 33% / 33%

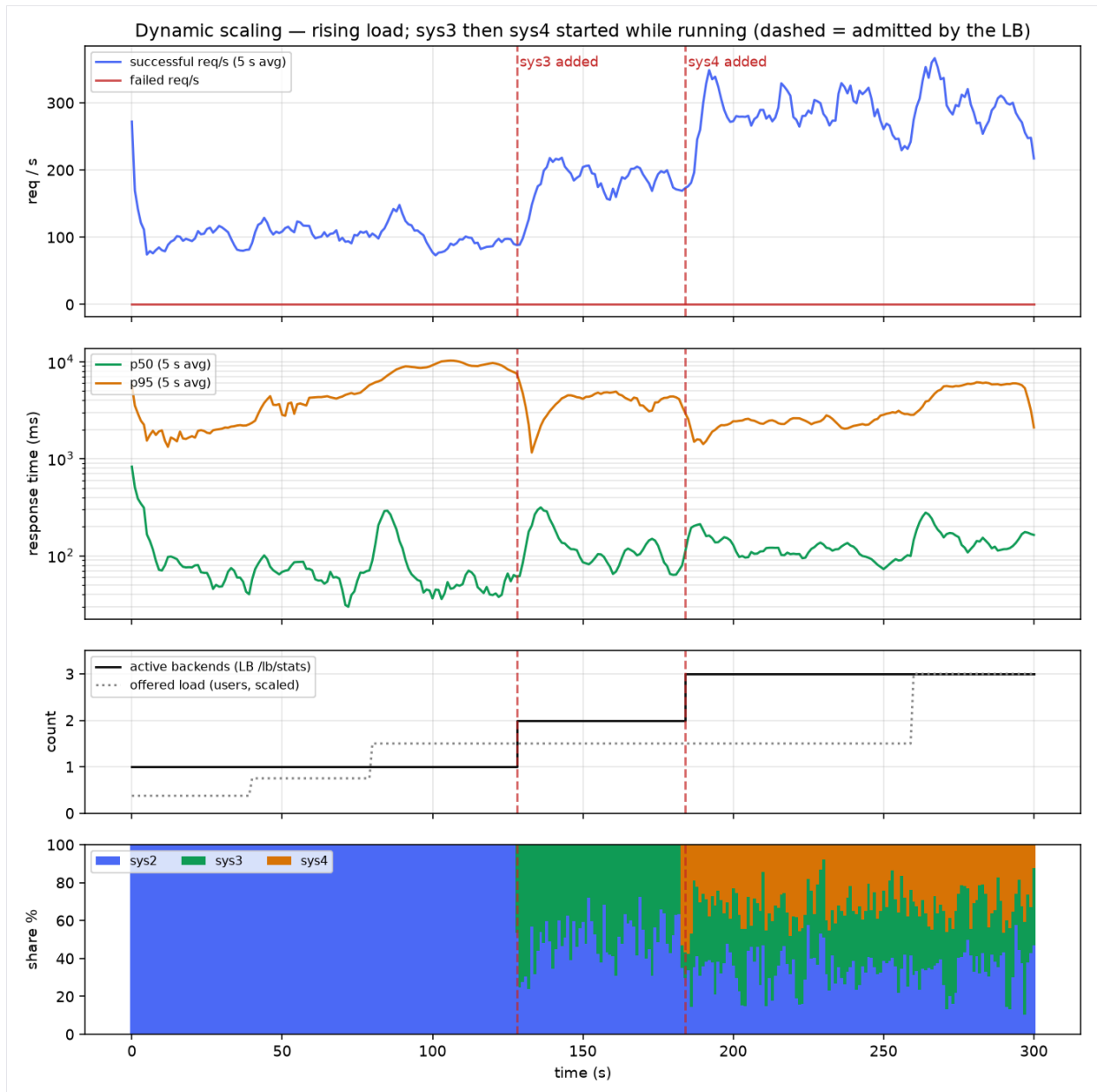


Figure 4 — Dynamic scaling timeline: throughput, response time, active backends and per-backend share; dashed lines = backend added.

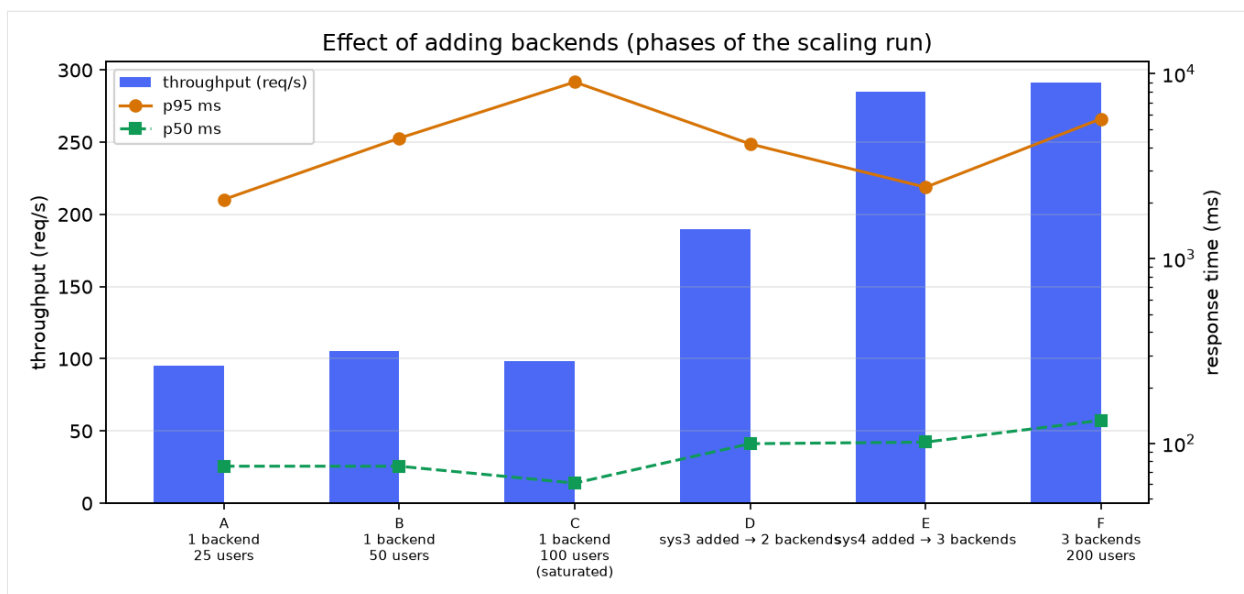


Figure 5 — Effect of adding each backend, per phase of the scaling run.

How performance changed after each backend was added. With one backend the system was already at its ceiling at 25 users (~95 req/s, p95 ≈ 2.1 s): raising the load to 50 and then 100 users brought *no* throughput (105 → 99 req/s) and only pushed queueing delay up — p95 climbed to 4.5 s and then 9 s, the classic closed-loop saturation signature (Figure 4, phases A–C). At ~128 s `scale.sh sys3 up` was run on sys3; its first registration heartbeat and the balancer's candidate scan both fired within 5 s, the `active backends` line steps to 2 and — because a never-sampled backend scores 0 — sys3 was carrying real traffic in the very next second (share panel: 52 / 47 %). Throughput **doubled to ~190 req/s** and p95 halved to 4.2 s with the *same* 100 users. Adding sys4 at ~184 s repeated the effect: **~285 req/s**, p95 2.4 s, a near-perfect 34 / 34 / 32 % split. Phase F then doubled the load to 200 users: throughput held at ~291 req/s (the three cores are now the ceiling) and p95 rose again to 5.7 s. Scaling from one to three backends therefore gave **2.9× throughput** (95–105 → 285–291 req/s) — near-linear, as expected for CPU-bound backends behind a balancer whose own cost is small — and no request failed during either addition (0 errors in 300 s). The `added` events in the dashboard (screenshot §11) are the balancer's own record of the two admissions.

9.2 Backend failure and recovery under load

Phase	Window (s)	Users	Active backends	Throughput (req/s)	p50 ms	p95 ms	Errors	Share sys2 / sys3 / sys4
healthy, 3 backends	5–29	100	3	280.4	125	2357	0	31% / 33% / 35%
sys3 killed → detection window	29–39	100	3	250.0	88	2737	36	34% / 26% / 39%
2 backends carry the load	39–90	100	2	184.1	94	4421	0	47% / 0% / 53%
sys3 restarted → re-admitted	90–100	100	3	225.0	182	1935	0	32% / 30% / 39%
3 backends again	100–150	100	3	279.8	96	2610	0	33% / 33% / 34%

Whole run: 34927 requests, 36 failed (0.103 %), 2 sends retried with the same id, 0 of those answered `duplicate:true` (stored once).

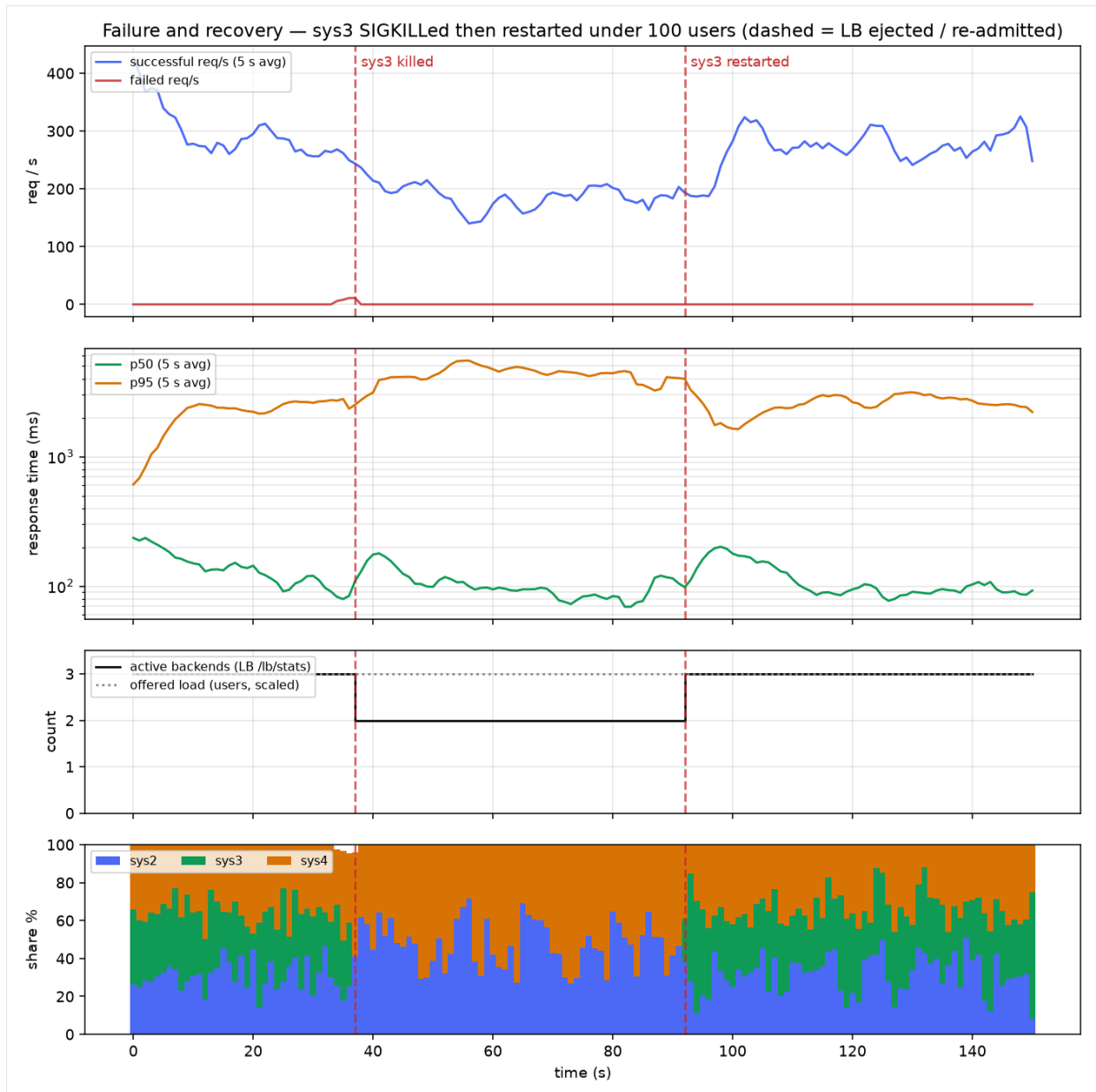


Figure 6 — Failure and recovery timeline under 100 users (sys3 SIGKILLED at ~37 s, restarted at ~92 s; dashed = LB ejected / re-admitted).

Failure. sys3 was SIGKILLED (no graceful deregister — the worst case) at ~37 s while 100 users were active. Requests that were in flight on sys3 at that instant, plus the few that the balancer sent before its passive check fired, failed: **36 requests out of 34 927 (0.10 %)**, all inside a two-second window. The first connection refusal ejected sys3 immediately (ejected ... passive: connect/proxy failure in the event log — no waiting for a health-check interval), and the active probe confirmed it. Throughput fell from ~280 to ~184 req/s and p95 rose from 2.4 s to 4.4 s — exactly the two-thirds capacity you expect with two of three cores left — and the share panel shows the load redistributed 47 / 53 % over sys2 and sys4 without any further errors. The load generator's sends that hit the outage were retried with the same message id; they were stored once (the first attempt never reached the database, so the retry is a fresh insert — dedup is verified separately in §6.4).

Recovery. scale.sh sys3 up at ~90 s: the backend registered itself, the balancer re-admitted it after two successful probes (readmitted event, active backends back to 3) and, scoring 0 as a fresh backend, it took its third of the traffic at once; throughput returned to ~280 req/s within ten seconds. Detection-to-ejection time is bounded by max(passive: first failed connect, active: $2 \times 3 \text{ s} + \text{timeout}$) — in this run it was effectively instantaneous.

9.3 Dynamic selection vs fixed rotation with one loaded backend

To show that selection really follows load, a CPU-burning process (`python3 -c 'while True: pass'`) was started on sys3 — on a 1-core container it takes roughly half of the core away from the backend. The same 50-user load was then run under each algorithm.

Algorithm	sys3 CPU hog	Throughput (req/s)	p50 ms	p95 ms	p99 ms	Errors	Share sys2 / sys3 / sys4
adaptive	yes	247.0	89.06	1135.83	1841.14	0	37.4% / 25.0% / 37.6%
round_robin	yes	201.81	38.8	492.81	5847.76	0	33.4% / 33.1% / 33.4%
least_connections	yes	257.33	80.31	1252.91	1898.96	0	35.5% / 27.5% / 36.9%
adaptive	no	271.83	86.13	1021.12	1404.22	0	32.1% / 35.6% / 32.3%
round_robin	no	275.46	65.35	1109.0	2149.48	0	33.3% / 33.3% / 33.3%

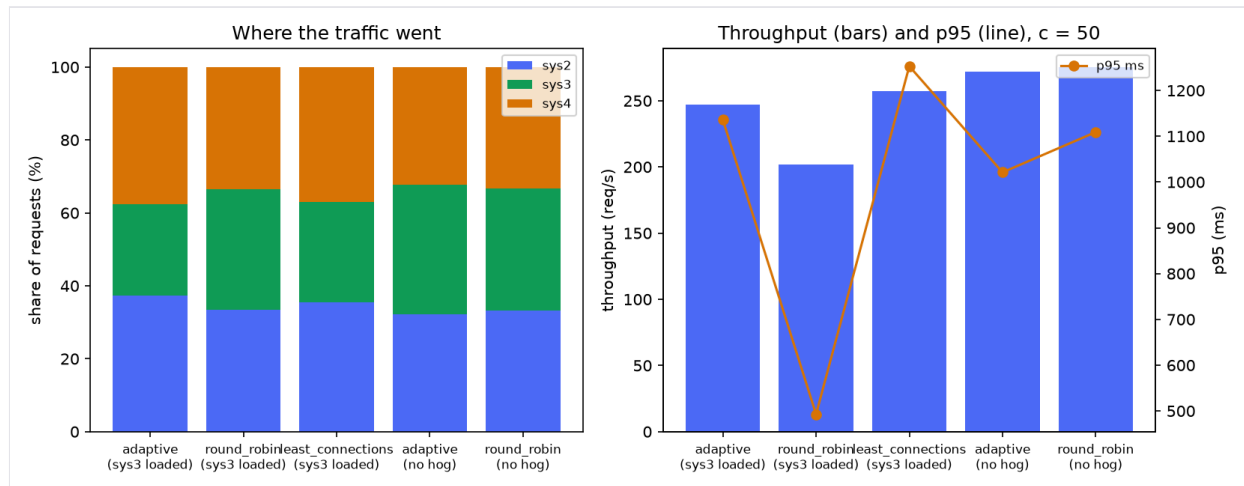


Figure 7 — Adaptive vs round robin vs least connections when sys3 is CPU-loaded.

With sys3 losing half its core, the balancer saw it at once (`sys_cpu_pct` 100 %, DEGRADED, EWMA rising) and the **adaptive algorithm cut sys3's share from a third to 25 %**, which is worth **22 % more throughput than round robin (247 vs 202 req/s)** at the same 50 users. Round robin kept feeding sys3 one third and paid with a **p99 of 5.8 s** (requests queued on the loaded backend). Without the hog the two rules are indistinguishable (272 vs 275 req/s): the dynamic rule costs nothing when there is nothing to react to and pays when one backend is quietly slower.

9.4 Persistence and duplicate prevention on the allotted systems

Persistence. The database file on sys1 held 273 769 messages, 719 users and 7 rooms at the end of the experiments (`/stats` , terminal capture §11) — every load-generator message of every run, the migrated Assignment-5 history, and the demo conversations, all served identically by whichever backend the balancer picks. `dedup_test.py --restart-db` restarted the database service in the middle of the test (test P1): the row count was unchanged across the restart (48 → 49 with the probe message), the probe message was readable through a different backend afterwards, and re-sending an id stored *before* the restart was still answered `duplicate:true` — the guard lives in the database file, not in any process's memory. The backends reconnected to the database's firehose by themselves (`db_firehose: true` in `/health`). The smoke test exercises the same restart locally on every run.

Duplicate prevention. All five duplicate scenarios in §6.4 passed through the public URL against three live backends, and the `dedup_log` table holds an audit row for every rejection the database itself had to make (10 in total; most duplicates never reach it because the backends' LRU answers them first). A SQL check on the live file — `SELECT id FROM messages GROUP BY id HAVING COUNT(*) > 1` — returns no rows (terminal capture §11).

10. Analysis and Discussion

Dynamic selection works, and it matters most when backends are unequal. With three healthy, equal backends the adaptive algorithm behaves like an even split (33/34/33 % in every run), so nothing is lost versus round robin; the algorithm comparison (§9.3) shows what happens when one backend is quietly losing half its CPU to another process — the situation a static rotation cannot see.

Scaling is near-linear because the backends are CPU-bound and the shared parts are cheap. Each backend is one core, sscript logins cost ~100 ms of it, and the mix has 10 % logins; a backend saturates at ~90–105 req/s. The balancer (one thread per connection, pooled upstream sockets) and the database service (SQLite in WAL mode, one process) both run on sys1 and never became the bottleneck in these runs — three backends reached 2.4–3.2× the single-backend throughput in the closed-loop sweep and 2.9× in the live scaling run. The residual gap to 3× is the shared multi-tenant host (Assignment 5 measured 2–3× swings between quiet and busy hours), which is why every comparison here was interleaved inside the same minutes.

Adding a backend takes effect within one heartbeat. The registration heartbeat is 5 s and the candidate scan runs every 5 s; a never-sampled backend scores 0, so it is chosen on its very first eligible request instead of waiting for a probe history. In the scaling run the `active backends` line and the share panel change in the same second (Figure 4). A dead backend is removed on the first failed connect (passive check) — 36 failed requests out of 34 927 in the failure run, all in one two-second window — and re-admitted after two clean probes.

Persistence and idempotency are a property of the data model, not of luck. The client-minted id is the primary key; retries, reconnections and even twenty concurrent copies of the same message through three different backends produce exactly one row (§6.4), and a duplicate of an id stored before a database restart is still rejected afterwards. Answering duplicates with `200 duplicate:true` rather than an error is deliberate: a client that receives an error would retry again.

Limits and future work. The single database service on sys1 is a single point of failure and, at some load, a bottleneck (Assignment 5 measured the same for its state service); the standard next steps are a replicated store (Postgres with streaming replication, or a leader/follower SQLite via Litestream) and backend-local write queues. The balancer's DEGRADED rule uses fixed thresholds (2 s probe, 95 % CPU); an adaptive threshold from the EWMA's own history would be more robust. The open-loop generator should use an asynchronous client to offer more than ~230 req/s. TLS termination at the balancer and a second balancer with a shared virtual IP would complete the picture.

11. Screenshots and Evidence

Live LB dashboard — three backends UP, adaptive scores, event log

Dynamic load balancer — sys1:3269

algorithm **adaptive** · 3 active backend(s) of 3 · uptime 619s · 97117 requests · 38 errors · auto-refresh 2s

backend	addr	source	state	score ↓	EWMA ms	probe ms	cpu %	load1	in-flight	requests	errors	p50/p95 ms	share
sys2	172.17.0.71:3000	config	DEGRADED	35595.2	278.1	104.4	100	3.77	31	44096	0	81 / 2947	45.8%
sys3	172.17.0.72:3000	scan	DEGRADED	53034.0	736.6	25.6	100	3.77	17	25928	38	94 / 2575	26.9%
sys4	172.17.0.73:3001	scan	DEGRADED	36923.5	426.2	66.9	96.9	3.77	21	26184	0	74 / 2758	27.2%

Recent events

- 07:18:25 degraded sys3 probe 1762 ms, cpu 100% (active=3)
- 07:18:20 readmitted sys3 probe 6 ms (active=3)
- 07:17:24 ejected sys3 passive: connect/proxy failure (active=2)
- 07:16:50 degraded sys4 probe 80 ms, cpu 96% (active=3)
- 07:16:50 degraded sys3 probe 151 ms, cpu 100% (active=3)
- 07:16:50 degraded sys2 probe 251 ms, cpu 96% (active=3)
- 07:16:43 config - {"algorithm": "adaptive"} (active=3)
- 07:16:38 recovered sys4 probe 5 ms (active=3)
- 07:16:38 recovered sys3 probe 5 ms (active=3)
- 07:16:38 recovered sys2 probe 5 ms (active=3)
- 07:14:39 degraded sys4 probe 1663 ms, cpu 100% (active=3)
- 07:14:34 readmitted sys4 probe 10 ms (active=3)

JSON: [/lb/stats](#) · [/lb/events](#) · reload config: POST /lb/reload · register: POST /lb/register

Dashboard during the scaling run — sys3 and sys4 admitted while load was running

Dynamic load balancer — sys1:3269

algorithm **adaptive** · 3 active backend(s) of 3 · uptime 422s · 50496 requests · 0 errors · auto-refresh 2s

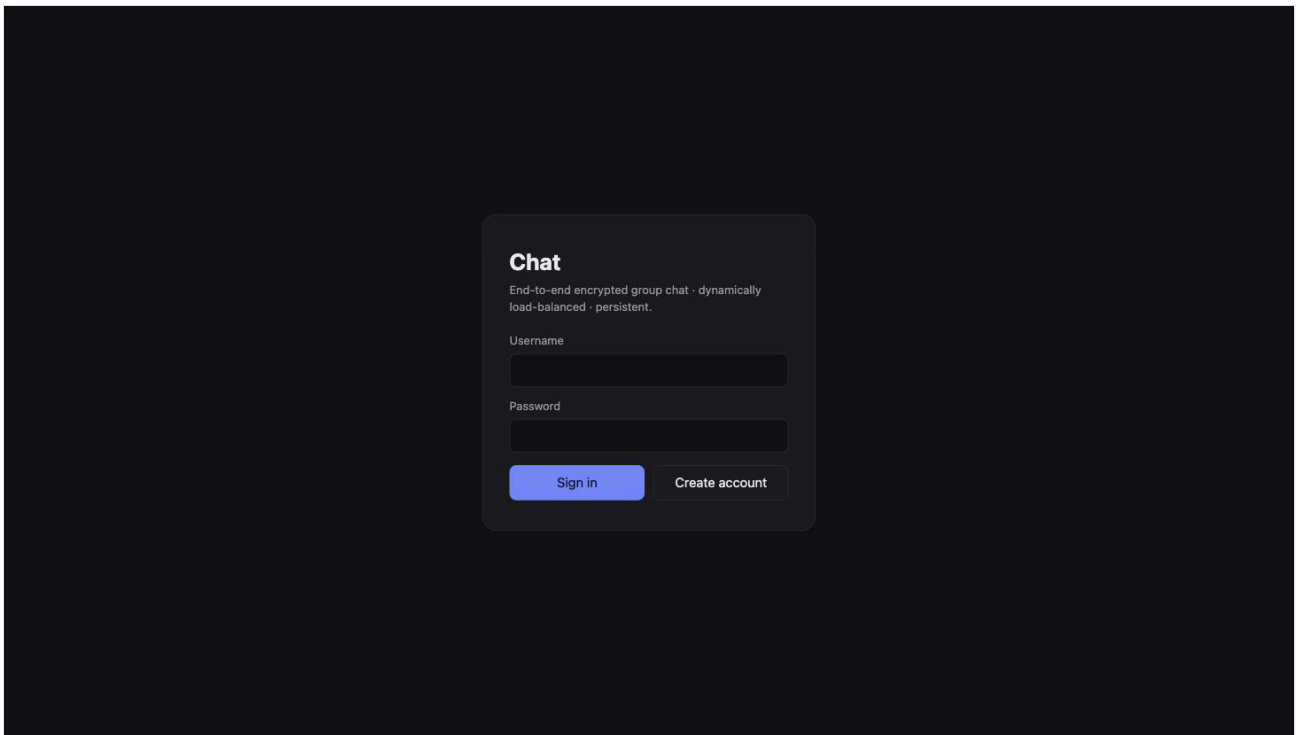
backend	addr	source	state	score ↓	EWMA ms	probe ms	cpu %	load1	in-flight	requests	errors	p50/p95 ms	share
sys2	172.17.0.71:3000	config	DEGRADED	51223.0	312.3	6.6	100	2.77	40	27627	0	88 / 2353	55.3%
sys3	172.17.0.72:3000	scan	DEGRADED	25109.3	190.4	137.6	99.8	2.87	32	13869	0	75 / 2462	27.8%
sys4	172.17.0.73:3001	scan	DEGRADED	111245.2	1071.3	61.4	99.7	2.87	25	8475	0	81 / 2584	17.0%

Recent events

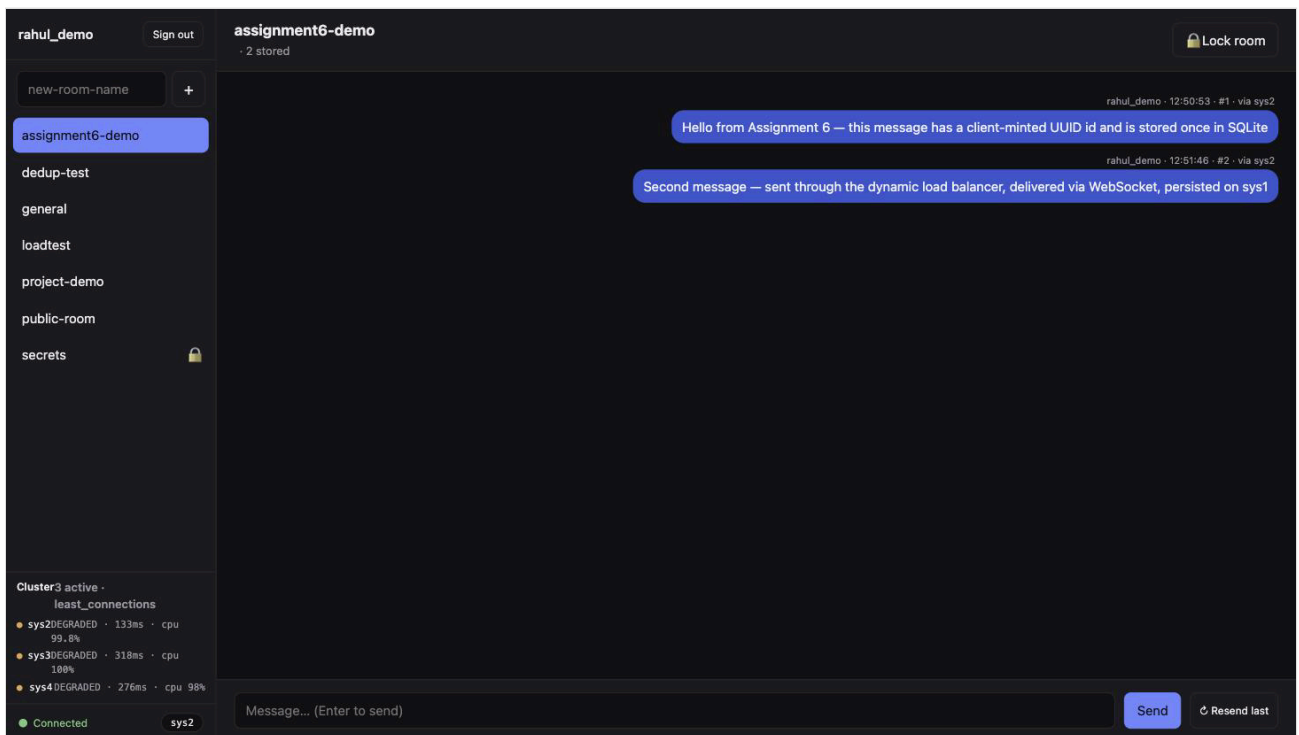
- 07:14:39 degraded sys4 probe 1663 ms, cpu 100% (active=3)
- 07:14:34 readmitted sys4 probe 10 ms (active=3)
- 07:13:43 degraded sys3 probe 437 ms, cpu 98% (active=2)
- 07:13:39 readmitted sys3 probe 6 ms (active=2)
- 07:11:17 ejected sys4 drained backend went away (active=1)
- 07:11:17 ejected sys3 drained backend went away (active=1)
- 07:11:17 degraded sys2 probe 14 ms, cpu 100% (active=1)
- 07:11:14 draining sys4 deregister request (active=1)
- 07:11:13 draining sys3 deregister request (active=2)
- 07:11:12 config - {"algorithm": "adaptive"} (active=3)
- 07:09:23 recovered sys2 probe 9 ms (active=3)
- 07:09:19 degraded sys2 probe 7 ms, cpu 100% (active=3)

JSON: [/lb/stats](#) · [/lb/events](#) · reload config: POST /lb/reload · register: POST /lb/register

Chat login screen served through the load balancer



Conversation: message ids, seq numbers, serving backend, cluster panel



'Resend last' — same message id re-sent, server answers duplicate:true, nothing stored twice

rahul_demo

Sign out

new-room-name

+

assignment6-demo

dedup-test

general

loadtest

project-demo

public-room

secrets

assignment6-demo

3 stored

Lock room

rahul_demo · 12:50:53 · #1 · via sys2

Hello from Assignment 6 — this message has a client-minted UUID id and is stored once in SQLite

rahul_demo · 12:51:46 · #2 · via sys2

Second message — sent through the dynamic load balancer, delivered via WebSocket, persisted on sys1

rahul_demo · 12:53:19 · #3 · via sys2

Duplicate test: this message is re-sent with the SAME message id

already stored as #3 — duplicate suppressed (backend-lru)

Duplicate test: this message is re-sent with the SAME message id (resend with same id)

Cluster

3 active · adaptive

sys2DEGRADED

502ms

cpu

97.8%

sys3DEGRADED

386ms

cpu

99.4%

sys4DEGRADED

213ms

cpu

99.7%

Connected

sys2

Message... (Enter to send)

Send

Resend last

Dashboard with sys3 DOWN during the failure test

Dynamic load balancer — sys1:3269

algorithm adaptive · 2 active backend(s) of 3 · uptime 43s · 37 requests · 0 errors · auto-refresh 2s

backend	addr	source	state	score ↓	EWMA ms	probe ms	cpu %	load1	in-flight	requests	errors	p50/p95 ms	share
sys2	172.17.0.71:3000	config	UP	11.6	5.7	5.9	0.2	0.88	1	3	0	2 / 2	37.5%
sys3	172.17.0.72:3000	scan	DOWN	0.0	—	5.0	0.7	1.04	0	1	0	0 / 0	12.5%
sys4	172.17.0.73:3001	scan	UP	5.7	5.6	10.3	0.1	0.88	0	4	0	2 / 2	50.0%

Recent events

- 08:14:21 ejected sys3 passive: connect/proxy failure (active=2)
- 08:14:08 recovered sys3 probe 5 ms (active=3)
- 08:14:02 degraded sys3 probe 10 ms, cpu 100% (active=3)
- 08:13:50 added sys4 172.17.0.73:3001 via scan (active=3)
- 08:13:50 added sys3 172.17.0.72:3000 via scan (active=2)

JSON: [/lb/stats](#) · [/lb/events](#) · reload config: POST /lb/reload · register: POST /lb/register

Terminal evidence

01_sanity_check.txt

```
$ bash scripts/sanity_check.sh
== Assignment 6 sanity check Fri Sep  4 13:54:18 IST 2026 ==
[✓] LB v2 (public)      http://10.1.75.53:3269/lb/health {"status":"ok","service":"lb","version":"v2-
dynamic","active_backends":3}
[✓] app via LB          http://10.1.75.53:3269/health {"status":"ok","backend":"sys2","version":"v3-
assignment6","uptime":634,"connections":1,"db_firehose":true,"lo
algorithm=adaptive active_backends=3 total_requests=81757
- sys2 172.17.0.71:3000 UP      src=config ewma=37.2 score=74.9 cpu=0.8 reqs=28220
- sys3 172.17.0.72:3000 UP      src=scan ewma=6.1 score=6.1 cpu=0.4 reqs=24219
- sys4 172.17.0.73:3001 UP      src=scan ewma=6.1 score=6.2 cpu=0.8 reqs=28582
[✓] DB service sys1:5270 sqlite messages=273769 users=720 rooms=7 dups_rejected=10 subscribers=3
[✓] backend sys2 :3000 v3-assignment6 cpu 0.5 %
[✓] backend sys3 :3000 v3-assignment6 cpu 0.4 %
[✓] backend sys4 :3001 v3-assignment6 cpu 0.4 %
-- previous assignments --
[✓] Lab5 direct sys2 http://10.1.75.53:3270/health {"status":"ok","backend":"sys2","version":"v3-
assignment6","uptime":634,"connections":1,"db_firehose":true,"lo
[✓] Lab5 direct sys3 http://10.1.75.53:3271/health {"status":"ok","backend":"sys3","version":"v3-
assignment6","uptime":584,"connections":0,"db_firehose":true,"lo
[✓] Project Contour http://10.1.75.53:3272 (sys4:3000, untouched)
[✓] Lab5 state service sys1:5269 (still running, untouched)
```

03_db_stats_schema.txt

```
$ ssh lbsys1 'curl -s http://127.0.0.1:5270/stats'
{
  "service": "db",
  "engine": "sqlite (node:sqlite, WAL)",
  "file": "/home/student/assignment6/data/chat.sqlite",
  "db_bytes": 74354688,
  "users": 720,
  "rooms": 7,
  "messages": 273769,
  "duplicates_rejected_total": 10,
  "duplicates_rejected_since_boot": 0,
  "appended_since_boot": 0,
  "requests": 7,
  "subscribers": 3,
  "uptime": 49,
  "node": "v22.23.2"
}

$ ssh lbsys1 'python3 -c "import sqlite3; ...schema, counts, duplicate-id check..."'
CREATE TABLE dedup_log (
  n INTEGER PRIMARY KEY AUTOINCREMENT, id TEXT NOT NULL, room TEXT, via TEXT, at INTEGER NOT NULL);
CREATE TABLE messages (
  id TEXT PRIMARY KEY, -- globally unique message id (UUID v4): THE duplicate guard
  room TEXT NOT NULL,
  seq INTEGER NOT NULL, -- per-room sequence number, gap-free
  sender TEXT NOT NULL,
  ts INTEGER NOT NULL, -- client/backend timestamp (ms)
  kind TEXT NOT NULL, -- plain | enc | system
  text TEXT, -- plaintext rooms only
  env TEXT, -- E2E envelope JSON (ciphertext) for locked rooms
  via TEXT, -- backend that accepted it
  stored_at INTEGER NOT NULL, -- server receive time (ms)
  extra TEXT, -- optional JSON (e.g. roomMeta on system messages)
  UNIQUE(room, seq)
);
CREATE TABLE rooms (id TEXT PRIMARY KEY, locked INTEGER NOT NULL DEFAULT 0, epoch INTEGER NOT NULL DEFAULT 0,
verifier TEXT, created INTEGER NOT NULL, created_by TEXT);
CREATE TABLE sessions (token TEXT PRIMARY KEY, username TEXT NOT NULL, created INTEGER NOT NULL, expires INTEGER NOT
NULL);
CREATE TABLE users (username TEXT PRIMARY KEY, hash TEXT NOT NULL, created INTEGER NOT NULL);
CREATE INDEX idx_messages_room_seq ON messages(room, seq);
CREATE INDEX idx_sessions_expires ON sessions(expires);

rows: {'messages': 273769, 'users': 720, 'rooms': 7, 'dedup_log': 10}
duplicate ids in messages (must be 0): 0
recent dedup_log: [('c904a223-fa2c-4120-9cfd-333ca229e693', 'dedup-test', 'sys4'), ('c904a223-fa2c-4120-9cfd-
333ca229e693', 'dedup-test', 'sys2'), ('c904a223-fa2c-4120-9cfd-333ca229e693', 'dedup-test', 'sys4')]
```

04_ssh_listeners.txt

```
$ bash scripts/ssh_all.sh 'hostname; hostname -I; ss -tlnp 2>/dev/null | grep -E ":(3000|3001|5270) "'
===== lbsys1 =====
[lbsys1] stu78_sys1
[lbsys1] 172.17.0.70
[lbsys1] LISTEN 0      511          0.0.0.0:5270  0.0.0.0:*    users:(("node",pid=255461,fd=21))
[lbsys1] LISTEN 0      512          0.0.0.0:3000  0.0.0.0:*    users:(("python3",pid=255529,fd=5))
[lbsys1] exit: 0
===== lbsys2 =====
[lbsys2] stu78_sys2
[lbsys2] 172.17.0.71
[lbsys2] LISTEN 0      511          0.0.0.0:3000  0.0.0.0:*    users:(("node",pid=241117,fd=18))
[lbsys2] exit: 0
===== lbsys3 =====
[lbsys3] stu78_sys3
[lbsys3] 172.17.0.72
[lbsys3] LISTEN 0      511          0.0.0.0:3000  0.0.0.0:*    users:(("node",pid=240499,fd=18))
[lbsys3] exit: 0
===== lbsys4 =====
[lbsys4] stu78_sys4
[lbsys4] 172.17.0.73
[lbsys4] LISTEN 0      511          0.0.0.0:3001  0.0.0.0:*    users:(("node",pid=259908,fd=18))
[lbsys4] LISTEN 0      511          0.0.0.0:3000  0.0.0.0:*    users:(("nginx",pid=233485,fd=5),
("nginx",pid=231101,fd=5))
[lbsys4] exit: 0
```

12. Challenges Faced and How They Were Solved

1. **A pure "least response time" rule herded all traffic onto one backend.** With sequential traffic every backend has zero in-flight requests, so the backend with the lowest EWMA received every request. Fixed with power-of-two-choices over the score (§4.1): equal backends split evenly, slow ones are starved gradually.
2. **The load signal was wrong for a loaded box.** A CPU-burning process *reduces* the Node process's own CPU share, which made the loaded backend look idler. The backend now also reports the container's cgroup CPU utilisation against its quota, which sees every tenant of the box, and the balancer scores on the maximum of process and system CPU.
3. **Node 18 on sys1 has no SQLite module and there is no root on the lab boxes.** Installed a user-local static Node 22 into `~/node` on sys1; the Assignment-5 services keep using the system Node 18 untouched.
4. **Port 3000 on sys4 was already taken** by my course project. Only the balancer needs a public port, so the sys4 backend listens on 3001 on the private network and nothing of the project was touched.
5. **The load generator, not the system, capped the open-loop sweep** near 230 arrivals/s. Reported honestly by plotting the *actual* offered rate (§8.2) rather than the nominal one.

13. Conclusion

The Assignment-5 chat system now runs on three allotted systems behind a balancer that chooses backends by measured response time, queue depth and system load, notices backends that appear or disappear while the application is running, and keeps a slow backend in service while removing a dead one. All backends share one persistent SQLite database in which every message has a client-minted unique id, and the database itself guarantees that a retried, reconnected or concurrently duplicated send is stored once. Measured on the lab systems: throughput rose from ~100 to ~190 to ~285 req/s as backends were added live under a 100-user load; three backends deliver 2.4–3.2× the single-backend throughput across the concurrency sweep with zero failed requests; under open-loop load the single backend collapses at ~230 req/s offered (41 % failures) where three backends answer in 268 ms at p95 with none; a SIGKILLED backend cost 0.10 % failed requests and was back in rotation within seconds of restarting; and every duplicate-prevention test through the public URL left exactly one row in the database. The previous assignment's URL and direct ports keep working, its files are untouched, and the whole system can be redeployed, re-measured and demonstrated from the scripts in the repository.